Klaro: what we built and what you must know

Study guide 1 of 3. Every number comes from the final report or the repo. Read this first.

1 The project in one minute

Problem. Companies bidding for contracts must read a tender document called an **RFP** (Request for Proposal), 50 to 100 pages long. Key facts are scattered. RFPs are confidential, so uploading them to ChatGPT-style services is often not allowed. Many are in German.

Solution. Klaro is a desktop app. You drop in an RFP PDF. A small AI model running on your own laptop reads it and writes a summary under **nine headings**. Every point shows its **page number**. Click it and the original PDF opens with the passage highlighted. German RFPs are translated to English on the laptop first. The summary can be shown in German. Dates can be saved as a calendar file. **Nothing leaves the laptop.**

Remember: Klaro = local, private, page-referenced RFP summary, English and German. Model: Qwen3.5 4B through Ollama. Desktop app: Electron + React. Backend: Python + FastAPI.

2 Parts of the system

Layer	What it is
`electron/`	Desktop shell. Main process starts the backend, the window shows the React screens (home, progress, results). Plain JavaScript.
`backend/api/`	FastAPI server on localhost. Routes: `/health`, `/runs`, summary, source PDF, evaluation, translation. Runs one job at a time, others wait in a queue.
`backend/` (pipeline)	Python code that does the real work: PDF reading, chunking, analysis, summary, evaluation, rendering.
`backend/translation/`	German detection, OPUS-MT translation, summary translation.
`scripts/finetune/`	Offline LoRA fine-tuning kit. Not part of the app.
`data/golden/`	45 RFPs and 32 reference summaries.

How a run goes. App starts local server → you add a PDF → server starts the pipeline as a background process (`run_rfp.py -progress-json`) → it prints JSON progress lines → the app shows a progress bar → results screen loads the summary HTML in an iframe beside the PDF.

3 The seven pipeline steps

1. **PDF extraction.** pypdf reads text page by page, keeps page numbers, skips contents pages.
2. **Chunking.** Pieces of about 1,800 words with 200 words overlap (about 4 pages), so a sentence cut at a boundary is not lost.
3. **Chunk analysis** (AI). Each piece is read *twice*: once for overview, scope, requirements, eligibility; once for dates, money, duration, legal conditions. Answers are a fixed JSON format with a page for each fact; Pydantic checks it; slightly bad output is repaired. If more than half of the chunks fail the run fails. *Why twice:* first try with one question gave 2 empty and 1 cut-off answer on 5 chunks.

4. **Aggregation.** Facts grouped by heading, repeats removed. Issuer = most frequent organisation name (Bank, Authority, Ministry endings); "Not identified" if unsure.
5. **Summary agent** (AI). Writes the nine headings from the combined facts. If a heading comes back empty, the code fills it by copying its facts (the "safety step").
6. **Evaluation.** Nine automatic checks: structure, grounding, traceability (page numbers), coverage, duplication, conciseness, hallucination, information loss, wrong heading.
7. **Rendering.** HTML summary with clickable page chips.

German runs add **translation** after step 1, making 8 stages.

Nine headings: Overview, Issuing organization, Scope, Key requirements, Eligibility, Important dates, Commercial requirements, Project duration, Important conditions.

Remember: Only steps 3 and 5 use the AI. The rest is plain code, fast and repeatable. About 75 to 95% of the time is step 3.

4 The German path

- **Detect language** offline by counting common German and English words. Too short or another language: the run stops and asks.
- **Translate** with OPUS-MT de-en (Helsinki-NLP, about 301 MB, offline) *segment by segment* (sentence or heading). Cached by hash of the text, so repeats are fast.
- **Number check.** Every number in the German segment must appear in the English one. If not, the segment is *flagged* (marked red in the app, never auto-fixed). Known weakness: times like 14:00 and two-digit years can trigger a false flag.
- The *unchanged English pipeline* then runs. German text never goes to the English prompts. Page numbers stay the physical PDF pages.
- **Evidence linking** (`evidence.json`). Facts only carry a page, and the summary agent rewrites them, so there is no stored link to a source sentence. After the summary, Klaro scores windows of 1 to 3 neighbouring segments on the cited page by word overlap (first 5 letters of each word) and keeps the best if score ≥ 0.6. Below 0.6 it claims no quote and says "exact passage not found on this page".
- **German view.** On request, the finished English summary is translated back with OPUS-MT en-de. Headings, order, page numbers, issuer name stay. Numbers, dates, currency signs are compared; differences flagged. Written to a staging folder and moved in at the end so a cancel leaves nothing.

5 Other features

Calendar export. Dates come from the "Important dates" bullets, plus other bullets with an event word (deadline, submission, opening; English or German). Understands month names in both languages, ISO dates, ranges, 12h/24h times. Numeric dates like 03/04/2026 are read day first and marked "assumed" when ambiguous. No year, no day or impossible date goes to "needs review". *Klaro never guesses a year.* Output is an `.ics` file. A "Google Calendar" button opens calendar.google.com in the browser, only on click, and sends that one event's title and date. We also added a top-bar button "Add all dates to calendar".

Security of the app. `contextIsolation` on, `nodeIntegration` off, sandbox on. The window can only ask the main process for a small fixed list of things (API address, file picker, read source PDF of the current run, save calendar). No `<webview>`. The summary iframe talks by `postMessage`; the app checks source, origin and that page numbers are integers in range. Only localhost network at run time. The only internet use: setup downloads (PyPI, huggingface), and the Google Calendar click.

Setup. `npm install` then `npm run dev`. A script finds Python 3.11+, makes `.venv`, installs pinned packages, downloads the translation models, checks Ollama. On a clean Mac: 93 s (74 s was the model download). Not run on Windows.

Config. All settings in `backend/config.py`, overridable by env vars: `KLARO_MODEL` (`qwen3.5:4b`), `KLARO_OLLAMA_URL`, `KLARO_DATA_DIR`, `KLARO_LLM_TIMEOUT` (300 s), `KLARO_PORT`, `KLARO_MODELS_DIR`, `KLARO_NUM_CTX` (0 = Ollama default).

6 Results you must know by heart

6.1 Model benchmark (20 chunks, 47 hand-marked requirements)

	Gemma 4 e4b	**Qwen3.5 4B**	Phi-4-mini
Found (of 47)	33	23	7
Wrong/extra items	22	**8**	9
Precision / recall	60 / 70	**74** / 49	44 / 15
F1	**65**	59	22
Mandatory vs optional correct	85	**91**	57
Download / params	9.6 GB / 8.0B	3.4 GB / 4.7B	2.5 GB / 3.8B

Why Qwen even though Gemma has higher F1: Qwen is more careful (74% precision, 8 wrong items vs 22), it is about a third of the download size, and a wrong point in a summary hurts more than a missing one, since Klaro asks two question sets per chunk to catch misses. Qwen's weak format following (0% exact) is handled by tolerant parsing.

6.2 Full runs on an Apple M4, 24 GB

	R1	R2	R3	G1	G2	G3
Lang	EN	EN	EN	DE	DE	DE
Pages/chunks	84/17	54/14	54/14	24/5	31/6	24/5
Time	22m51	25m19	22m16	9m04	10m30	15m21
Points	170	211	216	104	137	40
Facts included	99%	97%	96%	98.3%	96.8%	64.8%
Shorter than facts	8%	8%	9%	9.3%	9.9%	62.4%

R1 is a government RFP, R2 and R3 are the same 54-page bank RFP run twice (236 vs 243 facts: the model is not deterministic). G3 = G1 with `KLARO_NUM_CTX=32768`. G3 first attempt timed out after 787 s at the 300 s limit; retry with 900 s worked.

6.3 The context-size finding (our most interesting result)

- Ollama default context here is **4,096 tokens**. The summary step puts all facts in one prompt. In one test the facts were **17,496 tokens** but the model saw **2,050** (about 12%). The rest was cut off silently.
- Result: the model's answer was unusable, so the safety step *copied every fact in*. Summaries were only 8 to 10% shorter than the facts, and the "100% grounded" score was partly fake, since copied facts always match themselves.
- Fix: `KLARO_NUM_CTX`. On 11 golden documents: average summary 4,565 → 3,054 → **545 words** (default, 16384, 32768). Within limits 0 → 4 → 6 of 11. Duplicates 25.3 → 0.9. Grounding 100 → 94.1%. Summary step time 64 s → 194 s.
- **Cost / honest bits:** about three times slower; only 6 of 11 within limits; on the German run facts used fell from 98.3% to 64.8% (lost the buyer's name and address and some data protection and termination clauses). So *shorter is not automatically better*. Fix is **off by default**. We think grounding drops because the model now writes its own sentences and the check compares words, but we **did not test** that.

6.4 German numbers

Flags: G1 23 of 581 segments (4.0%), G2 4 of 589 (0.7%). Highlighting found a passage (≥0.6) for 90/104 (G1), 116/137 (G2), 26/40 (G3). English-to-German view of G1: 6.9 s for 110 segments, 1 bullet flagged (missing euro sign). Issuer line was wrong or partial in both German runs. G2 translation took 0.8 s (probably a cache hit, not checked).

7 Golden dataset and fine-tuning

- **45 English RFPs**, 8 sectors, 2,583 pages total (21 to 111 pages, median 57). **35 train, 10 test**, split by sector with a fixed seed.
- **32 summaries** so far: 22 train + all 10 test. Drafted by an AI, passed automatic checks, so called **auto-checked**. **None is human-verified.** Never call them verified.
- **LoRA kit** (`scripts/finetune/`): build dataset → train LoRA on the train split → merge → load in Ollama → score on the 10 test docs. The scoring script refuses a document used for training. Adapters saved as safetensors only (no pickle).
- Run only on a Mac with the **0.8B** model of the same family: loss 1.62 → 0.20 over 5 epochs, merged model loaded in Ollama. **Nothing trained on the real 4B.**
- **8 GB GPU limit.** Whole-summary examples are too long: median 34,597 tokens, only 1 of 32 under 14,000. Estimated 4-bit 4B memory: 14.7 GiB at 32k tokens, 7.8 GiB even at 6k (estimates, ±30%, no GPU used). So we switched to **per-section examples** (at most 4,096 tokens): 322 train, 39 validation, 165 test examples; about 7.3 GiB for 4B (borderline), 5.2 GiB for a 2B fallback.
- **Catch:** the app writes the whole summary in one call, so a per-section adapter cannot be used in the app yet. Future: section-by-section summary step.

8 Testing

68 JavaScript tests (54 for the calendar) and 43 Python tests pass. The app was driven by hand on macOS and screenshots are from the real app. **Not done:** automated tests of the real window, Windows testing.

9 Limitations (say these yourself before you are asked)

1. Golden summaries are not human-verified.
2. LoRA not trained or tested on the real 4B model, and not wired into the app.
3. No German speaker has judged translation quality; the A/B script in `scripts/translation_ab` was not run.
4. Windows untested.
5. Issuer line wrong or partial in both German runs; some points cite a page where the passage was not found.
6. Context fix is off by default, 3x slower, and lost facts on a German run.
7. Only text PDFs (no scanned), tables lose layout, 9 to 25 minutes per run.
8. Quality checks compare words with the extracted facts. They show agreement with the facts, not that extraction was right.
9. Benchmark is small (20 chunks, 47 requirements) and inherited from the earlier phase.

10 Future scope

Full training on the university supercomputer and evaluation on the 10 test docs; human verification of the golden set; section-by-section summary so the adapter can be used; scanned PDFs with OCR, Word files, more languages; a Proposal Assistant (requirements checklist, first proposal outline).

11 Who did what (for when you are asked)

Yasharth: desktop app, local server and settings file; German path (detection, translation step, evidence file, highlighting, English-to-German view); calendar export; one-command setup; golden dataset scripts; fine-tuning kit; this report. Nikunj: summary pipeline (chunk analysis, aggregation, summary agent), evaluator, renderer, command-line workflow. Shrey: PDF reader, chunker, section detector, baseline analyzer, requirement evaluation; later the first English-to-German summary translation. (From git history. Check your own part against your memory before the viva.)